

SEMINARARBEIT

Im Studiengang FH Technikum Wien, Bachelorstudiengang BIF
Lehrveranstaltung BIF-VZ-3-WS2025-CLCOM-EN/158471

Semesterprojekt – DevOps und Cloud Computing 3B

Ausgeführt von: Benjamin Feichtlbauer, Kevin Forter,
Mikulovic Luka
Tepsurkaev Abdulah

Wien, den 27. Januar 2026

Inhaltsverzeichnis

1	Milestone 2	1
1.1	Network Setup	1
1.1.1	VPC Configuration	1
1.1.2	Subnets	2
1.1.3	Internet Gateway	3
1.1.4	Route Tables	3
1.1.5	Security Groups	4
1.2	DNS Setup	7
1.2.1	Primary DNS	7
1.2.2	Secondary DNS	9
1.3	GitLab Server Setup	11
1.3.1	Installation	11
1.3.2	Verification	12
1.3.3	System Configuration	13
1.4	GitLab Runner Setup	13
1.4.1	Configuration	14
1.4.2	Registration	15
1.5	GitLab Workflow Verification	15
1.5.1	Repository Creation	15
1.5.2	SSH Configuration	17
1.5.3	Cloning and Committing	18
1.5.4	Pushing Changes	19
2	Milestone 3	21
2.1	LDAP Setup – Technical Documentation	21
2.1.1	Goal	21
2.1.2	LDAP EC2 Instance Creation	22
2.1.3	Security Configuration	23
2.1.4	Connection and Installation	23
2.1.5	Base Structure Configuration	24
2.1.6	User Creation	24
2.1.7	GitLab LDAP Integration	25
2.1.8	GitLab Configuration Adjustments	25
2.1.9	User Onboarding and Management	27

2.1.10	Result	32
2.2	CI/CD Pipeline Setup	32
2.2.1	Objective	32
2.2.2	Pipeline Decisions & Tools	33
2.2.3	Pipeline Stages	33
2.2.4	Environment & Configuration	33
2.2.5	Branching Strategy	34
2.3	Cost Estimation	34

1 Milestone 2

1.1 Network Setup

In this section, we describe the setup of the Virtual Private Cloud (VPC) and the associated network components within AWS.

1.1.1 VPC Configuration

We created a new VPC with the IPv4 CIDR block `10.0.0.0/16`. This provides a private network space for our infrastructure.

The screenshot shows the 'VPC settings' section of the AWS VPC console. It includes options for 'Resources to create' (VPC only), 'Name tag' (lph-vpc), 'IPv4 CIDR block' (10.0.0.0/16), 'IPv6 CIDR block' (No IPv6 CIDR block), 'Tenancy' (Default), and 'VPC encryption control' (None). Below this is a 'Tags' section with a key 'Name' and value 'lph-vpc'.

VPC settings

Resources to create [Info](#)
Create only the VPC resource or the VPC and other networking resources.

☒ VPC only ☐ VPC and more

Name tag - optional
Creates a tag with a key of 'Name' and a value that you specify.

lph-vpc

IPv4 CIDR block [Info](#)
☒ IPv4 CIDR manual input
☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR
10.0.0.0/16
CIDR block size must be between /16 and /28.

IPv6 CIDR block [Info](#)
☒ No IPv6 CIDR block
☐ IPAM-allocated IPv6 CIDR block
☐ Amazon-provided IPv6 CIDR block
☐ IPv6 CIDR owned by me

Tenancy [Info](#)
Default

VPC encryption control (\$) - new [Info](#)
Monitor mode provides visibility into encryption status without blocking traffic. Enforce mode prevents unencrypted traffic. [Additional charges apply](#)

☒ None ☐ Monitor mode
See which resources in your VPC are unencrypted but allow the creation of unencrypted resources. ☐ Enforce mode
Requires all resources, except exclusions, in your VPC to be encryption-capable and blocks creation of unencrypted resources.

Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key **Value - optional**

Q Name X Q lph-vpc X [Remove tag](#)

[Add tag](#)
You can add 49 more tags

Abbildung 1: VPC Configuration

1.1.2 Subnets

A public subnet was created within the VPC with the CIDR block `10.0.0.0/24`. This subnet hosts our instances and allows them to communicate with each other and the internet.

Create subnet [Info](#)

VPC
VPC ID
Create subnets in this VPC.
vpc-0456ae6dc8078ccee (lph-vpc) ▼

Associated VPC CIDRs
IPv4 CIDRs
10.0.0.0/16

Subnet settings
Specify the CIDR blocks and Availability Zone for the subnet.

Subnet 1 of 1
Subnet name
Create a tag with a key of 'Name' and a value that you specify.
lph-subnet
The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.
United States (N. Virginia) / use1-az1 (us-east-1a) ▼

IPv4 VPC CIDR block [Info](#)
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.
10.0.0.0/16 ▼

IPv4 subnet CIDR block
10.0.0.0/24 256 IPs
< > ^ v

Tags - optional
Remove

Add new subnet

Cancel Create subnet

Abbildung 2: Subnet Configuration

Edit subnet settings [Info](#)

Subnet

Subnet ID: [subnet-04a0418d89832580c](#) Name: [lph-subnet](#)

Auto-assign IP settings [Info](#)
Enable AWS to automatically assign a public IPv4 or IPv6 address to a new primary network interface for an instance in this subnet.

☒ Enable auto-assign public IPv4 address [Info](#)

☐ Enable auto-assign customer-owned IPv4 address [Info](#)
Option disabled because no customer owned pools found.

Resource-based name (RBN) settings [Info](#)
Specify the hostname type for EC2 instances in this subnet and optional RBN DNS query settings.

☐ Enable resource name DNS A record on launch [Info](#)

☐ Enable resource name DNS AAAA record on launch [Info](#)

Hostname type [Info](#)

☐ Resource name

☒ IP name

DNS64 settings
Enable DNS64 to allow IPv6-only services in Amazon VPC to communicate with IPv4-only services and networks.

☐ Enable DNS64 [Info](#)

[Cancel](#) [Save](#)

Abbildung 3: Subnet Settings

1.1.3 Internet Gateway

To enable internet access for the resources in our public subnet, we attached an Internet Gateway to the VPC.

[VPC](#) > [Internet gateways](#) > Create internet gateway

Create internet gateway [Info](#)
An internet gateway is a virtual router that connects a VPC to the internet. To create a new internet gateway specify the name for the gateway below.

Internet gateway settings

Name tag
Creates a tag with a key of 'Name' and a value that you specify.

Tags - optional
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional	
Name	lph-igw	Remove

[Add new tag](#)
You can add 49 more tags.

[Cancel](#) [Create internet gateway](#)

Abbildung 4: Internet Gateway

1.1.4 Route Tables

A custom route table was configured to direct traffic destined for the internet (0.0.0.0/0) to the Internet Gateway. This route table is associated with our public subnet.

Create route table [info](#)

A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.

Route table settings

Name - optional

Create a tag with a key of 'Name' and a value that you specify.

lph-rt

VPC

The VPC to use for this route table.

vpc-0456ae6dc8078ccee (lph-vpc)

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Q Name

Value - optional

Q lph-rt

Remove

Add new tag

You can add 49 more tags.

Cancel

Create route table

Abbildung 5: Route Table

Edit routes

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	CreateRouteTable
Q 0.0.0.0/0	Q local	-	No	CreateRoute
	Internet Gateway			
	Q igw-0f6cba63cc8b3d95f			

Add route

Cancel

Preview

Save changes

Abbildung 6: Route Table Default Route

Edit subnet associations

Change which subnets are associated with this route table.

Available subnets (1/1)

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
☒ lph-subnet	subnet-04a0418d89832580c	10.0.0.0/24	-	Main (rtb-0c7f5277cb71b4682)

Selected subnets

subnet-04a0418d89832580c / lph-subnet

Cancel

Save associations

Abbildung 7: Route Table Subnet Association

1.1.5 Security Groups

Specific security groups were created for each component to control inbound and outbound traffic.

Create security group [Info](#)

A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

Basic details

Security group name [Info](#)

Name cannot be edited after creation.

Description [Info](#)

VPC [Info](#)

Inbound rules [Info](#)

Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info	
DNS (UDP)	UDP	53	Custom	Q 10.0.0.0/16	Delete
				10.0.0.0/16	
DNS (TCP)	TCP	53	Custom	Q 10.0.0.0/16	Delete
				10.0.0.0/16	
SSH	TCP	22	Custom	Q 84.115.226.143/32	Delete
				84.115.226.143/32	

Outbound rules [Info](#)

Type Info	Protocol Info	Port range Info	Destination Info	Description - optional Info	
All traffic	All	All	Custom	Q	Delete
				0.0.0.0/0	

Rules with destination of 0.0.0.0/0 or ::/0 allow your instances to send traffic to any IPv4 or IPv6 address. We recommend setting security group rules to be more restrictive and to only allow traffic to specific known IP addresses.

Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

No tags associated with the resource.

You can add up to 50 more tags

[Cancel](#)

Abbildung 8: DNS Security Group

Create security group [Info](#)

A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

Basic details

Security group name [Info](#)

lph-gitlab-sg

Name cannot be edited after creation.

Description [Info](#)

Allows public access to GitLab (HTTP/HTTPS) and internal DNS queries; SSH only fo

VPC [Info](#)

vpc-0456ae6dc8078ccee (lph-vpc)

Inbound rules [Info](#)

Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info	
HTTP	TCP	80	Anywh...	0.0.0.0/0	Delete
HTTPS	TCP	443	Anywh...	0.0.0.0/0	Delete
DNS (UDP)	UDP	53	Custom	10.0.0.0/16	Delete
DNS (TCP)	TCP	53	Custom	10.0.0.0/16	Delete
SSH	TCP	22	Custom	84.115.226.143/32	Delete

Add rule

Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

Outbound rules [Info](#)

Type Info	Protocol Info	Port range Info	Destination Info	Description - optional Info	
All traffic	All	All	Custom	0.0.0.0/0	Delete

Add rule

Rules with destination of 0.0.0.0/0 or ::/0 allow your instances to send traffic to any IPv4 or IPv6 address. We recommend setting security group rules to be more restrictive and to only allow traffic to specific known IP addresses.

Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

No tags associated with the resource.

Add new tag

You can add up to 50 more tags

Cancel

Create security group

Abbildung 9: GitLab Security Group

Create security group [Info](#)

A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

Basic details

Security group name [Info](#)

Name cannot be edited after creation.

Description [Info](#)

VPC [Info](#)

Inbound rules [Info](#)

Type	Protocol	Port range	Source	Description - optional	
HTTP	TCP	80	Custom	Q sg-01e7e340b978df65c	Delete
DNS (UDP)	UDP	53	Custom	Q sg-01e7e340b978df65c	Delete
DNS (TCP)	TCP	53	Custom	Q 10.0.0.0/16	Delete
SSH	TCP	22	My IP	Q 10.0.0.0/16	Delete
				Q 84.115.226.143/32	

[Add rule](#)

Outbound rules [Info](#)

Type	Protocol	Port range	Destination	Description - optional	
All traffic	All	All	Custom	Q 0.0.0.0/0	Delete

[Add rule](#)

Rules with destination of 0.0.0.0/0 or ::/0 allow your instances to send traffic to any IPv4 or IPv6 address. We recommend setting security group rules to be more restrictive and to only allow traffic to specific known IP addresses.

Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

No tags associated with the resource.

[Add new tag](#)

You can add up to 50 more tags

[Cancel](#) [Create security group](#)

Abbildung 10: Runner Security Group

1.2 DNS Setup

We implemented a split-horizon DNS architecture using Bind9, with a primary and a secondary DNS server to ensure redundancy. The domain configured is `ci.intern`.

1.2.1 Primary DNS

The primary DNS server is hosted on an EC2 instance with the private IP `10.0.0.10`.

i-0d67bc652ad85ca6c (primary.dns.ci.intern)		
Details	Status and alarms	Monitoring Security Networking Storage Tags
▼ Instance summary Info		
Instance ID i-0d67bc652ad85ca6c	Public IPv4 address 3.239.108.116 open address	Private IPv4 addresses 10.0.0.10
IPv6 address -	Instance state Running	Public DNS -
Hostname type IP name: ip-10-0-0-10.ec2.internal	Private IP DNS name (IPv4 only) ip-10-0-0-10.ec2.internal	
Answer private resource DNS name -	Instance type t3.micro	Elastic IP addresses -
Auto-assigned IP address 3.239.108.116 [Public IP]	VPC ID vpc-0456gae6dc8078ccce (lph-vpc)	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
IAM Role -	Subnet ID subnet-04a0418d89832580c (lph-subnet)	Auto Scaling Group name -
IMDSv2 Optional ⚠ EC2 recommends setting IMDSv2 to required Learn more	Instance ARN arn:aws:ec2:us-east-1:302123595647:instance/i-0d67bc652ad85ca6c	Managed false
Operator -		
▼ Instance details Info		
AMI ID ami-0c398cb65a93047f2	Monitoring disabled	Platform details Linux/UNIX
AMI name ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-20251015	Allowed image -	Termination protection Disabled
Stop protection Disabled	Launch time Sat Dec 06 2025 22:41:29 GMT+0100 (Mitteleuropäische Normalzeit) (7 minutes)	AMI location amazon/ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-20251015
Instance reboot migration Default (On)	Instance auto-recovery Default	Lifecycle normal
Stop-hibernate behavior Disabled	AMI Launch index 0	Key pair assigned at launch -

Abbildung 11: Primary DNS Instance Details

Installation

We installed Bind9 on the Ubuntu 22.04 LTS instance:

```
sudo apt update
sudo apt install bind9 bind9utils -y
```

Configuration

The zone configuration in `/etc/bind/named.conf.local` defines the forward and reverse lookup zones:

```
zone "ci.intern" {
    type master;
    file "/etc/bind/db.ci.intern";
    allow-transfer { 10.0.0.11; };
};

zone "0.0.10.in-addr.arpa" {
    type master;
```

```
file "/etc/bind/db.10";
allow-transfer { 10.0.0.11; };
};
```

The forward zone file `/etc/bind/db.ci.intern` maps hostnames to IP addresses:

```
$TTL 3600
@ IN SOA primary.dns.ci.intern. admin.ci.intern. (
    2025010101 ; Serial
    3600       ; Refresh
    1800       ; Retry
    604800     ; Expire
    86400 )    ; Negative Cache TTL

    IN NS primary.dns.ci.intern.
    IN NS secondary.dns.ci.intern.

primary.dns IN A 10.0.0.10
secondary.dns IN A 10.0.0.11
gitlab IN A 10.0.0.20
runner IN A 10.0.0.21
```

The reverse zone file `/etc/bind/db.10` maps IP addresses back to hostnames:

```
$TTL 3600
@ IN SOA primary.dns.ci.intern. admin.ci.intern. (
    2025010101
    3600
    1800
    604800
    86400 )

    IN NS primary.dns.ci.intern.
    IN NS secondary.dns.ci.intern.

10 IN PTR primary.dns.ci.intern.
11 IN PTR secondary.dns.ci.intern.
20 IN PTR gitlab.ci.intern.
21 IN PTR runner.ci.intern.
```

1.2.2 Secondary DNS

The secondary DNS server is hosted on an EC2 instance with the private IP `10.0.0.11`. It acts as a slave to the primary server.

i-002558c0145a1ee9b (secondary.dns.ci.intern)		
Details Status and alarms Monitoring Security Networking Storage Tags		
▼ Instance summary Info		
Instance ID i-002558c0145a1ee9b	Public IPv4 address 34.201.9.150 open address	Private IPv4 addresses 10.0.0.11
IPv6 address —	Instance state ✔ Running	Public DNS —
Hostname type IP name: ip-10-0-0-11.ec2.internal	Private IP DNS name (IPv4 only) ip-10-0-0-11.ec2.internal	Elastic IP addresses —
Answer private resource DNS name —	Instance type t3.micro	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto-assigned IP address 34.201.9.150 [Public IP]	VPC ID vpc-0456ae6dc8078ccee (lph-vpc)	Auto Scaling Group name —
IAM Role —	Subnet ID subnet-04a0418d89832580c (lph-subnet)	Managed false
IMDSv2 Optional EC2 recommends setting IMDSv2 to required Learn more	Instance ARN arn:aws:ec2:us-east-1:302123595647:instance/i-002558c0145a1ee9b	
Operator —		
▼ Instance details Info		
AMI ID ami-0c398cb65a93047f2	Monitoring disabled	Platform details Linux/UNIX
AMI name ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-20251015	Allowed image —	Termination protection Disabled
Stop protection Disabled	Launch time Sat Dec 06 2025 23:18:29 GMT+0100 (Mitteleuropäische Normalzeit) (less than a minute)	AMI location amazon/ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-20251015
Instance reboot migration Default (On)	Instance auto-recovery Default	Lifecycle normal
Stop-hibernate behavior —	AMI Launch index —	Key pair assigned at launch —

Abbildung 12: Secondary DNS Instance Details

Configuration

The configuration in `/etc/bind/named.conf.local` specifies the master server for zone transfers:

```
zone "ci.intern" {
    type slave;
    masters { 10.0.0.10; };
    file "/var/cache/bind/db.ci.intern";
};

zone "0.0.10.in-addr.arpa" {
    type slave;
    masters { 10.0.0.10; };
    file "/var/cache/bind/db.10";
};
```

1.3 GitLab Server Setup

We deployed a self-hosted GitLab CE server using Docker on an EC2 instance with the private IP 10.0.0.20.

The screenshot displays the AWS Management Console for an EC2 instance. The top navigation bar shows tabs for Details, Status and alarms, Monitoring, Security, Networking, Storage, and Tags. The 'Details' tab is selected, showing the instance summary and details. The instance is named 'i-0ae3da6c70d14ad10 (gitlab.ci.intern)' and is in a 'Running' state. The summary section includes fields for Instance ID, IPv6 address, Hostname type, Answer private resource DNS name, Auto-assigned IP address, IAM Role, IMDSv2, and Operator. The details section includes fields for AMI ID, AMI name, Stop protection, Instance reboot migration, Stop-hibernate behavior, Monitoring, Allowed image, Launch time, Instance auto-recovery, AMI Launch index, Platform details, Termination protection, AMI location, Lifecycle, and Key pair assigned at launch.

Instance summary		
Instance ID	Public IPv4 address	Private IPv4 addresses
i-0ae3da6c70d14ad10	44.211.119.180 open address	10.0.0.20
IPv6 address	Instance state	Public DNS
-	Running	-
Hostname type	Private IP DNS name (IPv4 only)	Elastic IP addresses
IP name: ip-10-0-0-20.ec2.internal	ip-10-0-0-20.ec2.internal	-
Answer private resource DNS name	Instance type	AWS Compute Optimizer finding
-	t3.medium	Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto-assigned IP address	VPC ID	Auto Scaling Group name
44.211.119.180 [Public IP]	vpc-0456ae6dc8078ccee (lph-vpc)	-
IAM Role	Subnet ID	Managed
-	subnet-04a0418d89832580c (lph-subnet)	false
IMDSv2	Instance ARN	
Optional	arn:aws:ec2:us-east-1:302123595647:instance/i-0ae3da6c70d14ad10	
EC2 recommends setting IMDSv2 to required Learn more		
Operator		
-		
Instance details		
AMI ID	Monitoring	Platform details
ami-0c398cb65a93047f2	disabled	Linux/UNIX
AMI name	Allowed image	Termination protection
ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-20251015	-	Disabled
Stop protection	Launch time	AMI location
Disabled	Sun Dec 07 2025 14:33:32 GMT+0100 (Mitteleuropäische Normalzeit) (7 minutes)	amazon/ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-20251015
Instance reboot migration	Instance auto-recovery	Lifecycle
Default (On)	Default	normal
Stop-hibernate behavior	AMI Launch index	Key pair assigned at launch
Disabled	-	-

Abbildung 13: GitLab Server Instance Details

1.3.1 Installation

Docker and Docker Compose were installed, and a persistent volume structure was created. The `docker-compose.yml` configuration is as follows:

```
version: '3.7'
services:
  gitlab:
    image: gitlab/gitlab-ce:latest
    hostname: "gitlab.ci.intern"
    restart: always
    container_name: gitlab
    ports:
      - "80:80"
      - "443:443"
```

```
volumes:
  - /srv/gitlab/config:/etc/gitlab
  - /srv/gitlab/logs:/var/log/gitlab
  - /srv/gitlab/data:/var/opt/gitlab
environment:
  GITLAB_OMNIBUS_CONFIG: |
    external_url 'http://gitlab.ci.intern'
```

1.3.2 Verification

The GitLab instance is accessible via the browser using the public IP or the internal DNS name (within the network).

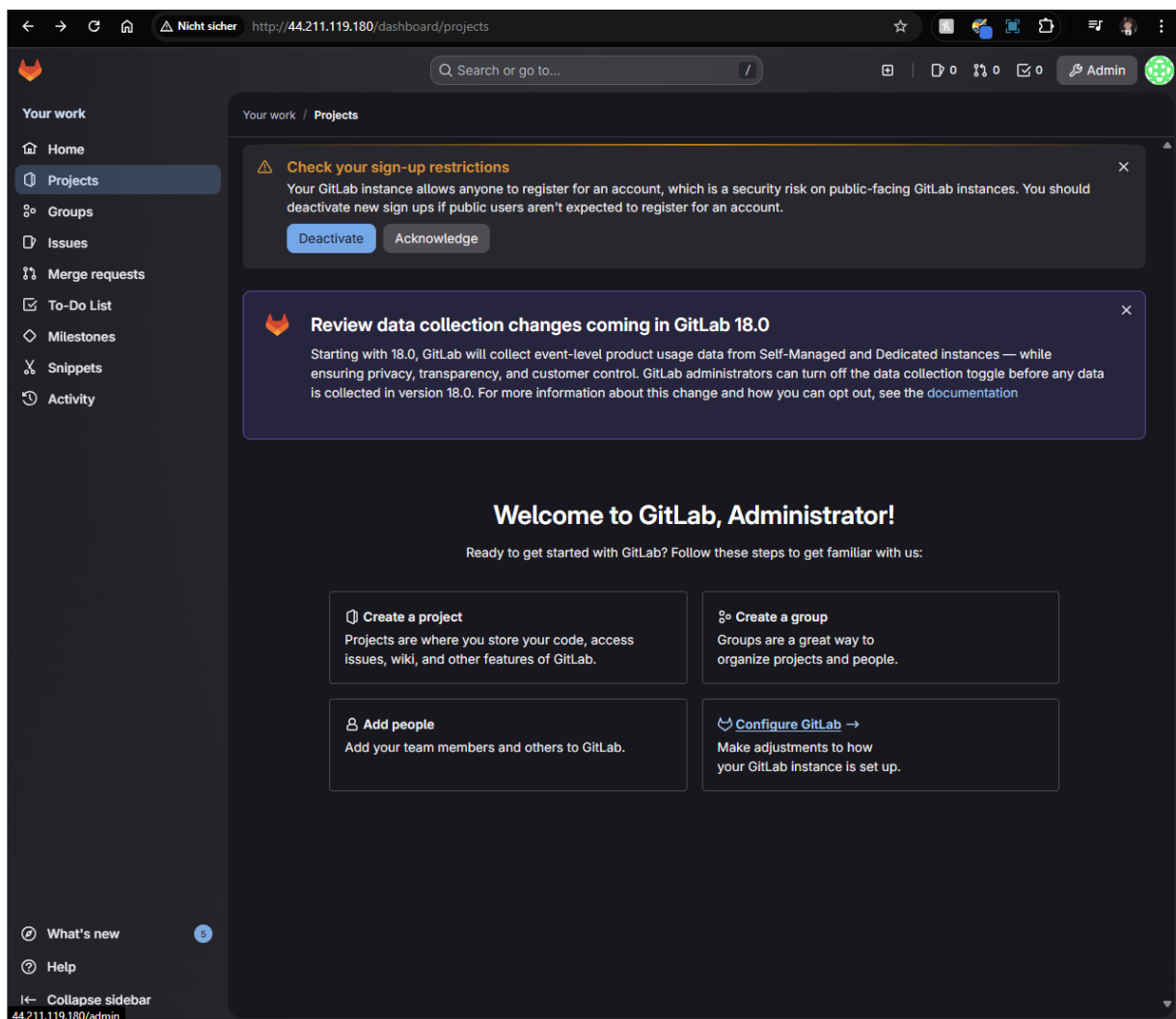


Abbildung 14: GitLab Homepage

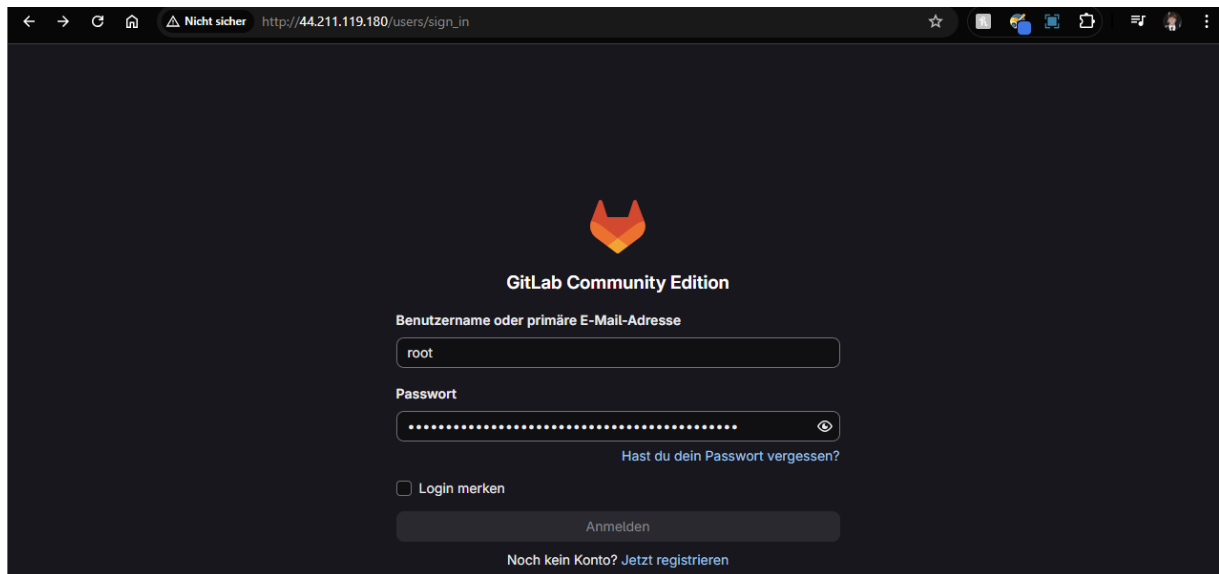


Abbildung 15: GitLab Login

1.3.3 System Configuration

The server runs on a `t3.medium` instance with 20 GB of gp3 storage to ensure sufficient resources for GitLab.

Swap Configuration

To prevent out-of-memory errors, we configured a 4GB swap file:

```
sudo fallocate -l 4G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile_none_swap_sw_0_0' | sudo tee -a /etc/fstab
```

Initial Access

After installation, the initial root password was retrieved from the container configuration:

```
sudo cat /srv/gitlab/config/initial_root_password
```

1.4 GitLab Runner Setup

A GitLab Runner was set up on a separate EC2 instance (`10.0.0.21`) to execute CI/CD jobs.

i-0ed503e5805fdc68b (runner.ci.intern)		
Details Status and alarms Monitoring Security Networking Storage Tags		
▼ Instance summary Info		
Instance ID i-0ed503e5805fdc68b	Public IPv4 address 3.237.191.41 open address	Private IPv4 addresses 10.0.0.21
IPv6 address -	Instance state Running	Public DNS -
Hostname type IP name: ip-10-0-0-21.ec2.internal	Private IP DNS name (IPv4 only) ip-10-0-0-21.ec2.internal	Elastic IP addresses -
Answer private resource DNS name -	Instance type t3.micro	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto-assigned IP address 3.237.191.41 [Public IP]	VPC ID vpc-0456ae6dc8078ccee (lph-vpc)	Auto Scaling Group name -
IAM Role -	Subnet ID subnet-04a0418d89832580c (lph-subnet)	Managed false
IMDSv2 Optional EC2 recommends setting IMDSv2 to required Learn more	Instance ARN arn:aws:ec2:us-east-1:302123595647:instance/i-0ed503e5805fdc68b	
Operator -		
▼ Instance details Info		
AMI ID ami-0c398cb65a93047f2	Monitoring disabled	Platform details Linux/UNIX
AMI name ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-20251015	Allowed image -	Termination protection Disabled
Stop protection Disabled	Launch time Sun Dec 07 2025 17:04:57 GMT+0100 (Mitteleuropäische Normalzeit) (10 minutes)	AMI location amazon/ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-20251015
Instance reboot migration Default (On)	Instance auto-recovery Default	Lifecycle normal
Stop-hibernate behavior -	AMI Launch index -	Key pair assigned at launch -

Abbildung 16: GitLab Runner Instance Details

1.4.1 Configuration

The runner was configured to use the internal DNS servers to resolve the GitLab server's hostname.

```
sudo systemctl disable --now systemd-resolved
sudo rm /etc/resolv.conf
echo "nameserver_10.0.0.10" | sudo tee /etc/resolv.conf
echo "nameserver_10.0.0.11" | sudo tee -a /etc/resolv.conf
```

The runner container was started with specific DNS settings:

```
sudo docker run -d --name gitlab-runner \
  --restart always \
  -v /srv/gitlab-runner/config:/etc/gitlab-runner \
  --dns 10.0.0.10 \
  --dns 10.0.0.11 \
  gitlab/gitlab-runner:latest
```

1.4.2 Registration

The runner was registered with the GitLab server using the internal URL `http://gitlab.ci.intern`.

```
sudo docker exec -it gitlab-runner gitlab-runner register
# URL: http://gitlab.ci.intern
# Executor: docker
# Default Docker image: alpine:latest
```

1.5 GitLab Workflow Verification

To ensure the GitLab server and Runner are functioning correctly, we performed a complete workflow test involving repository creation, code changes, and pipeline execution.

1.5.1 Repository Creation

We created a new blank project named `test-runner` in the GitLab web interface.

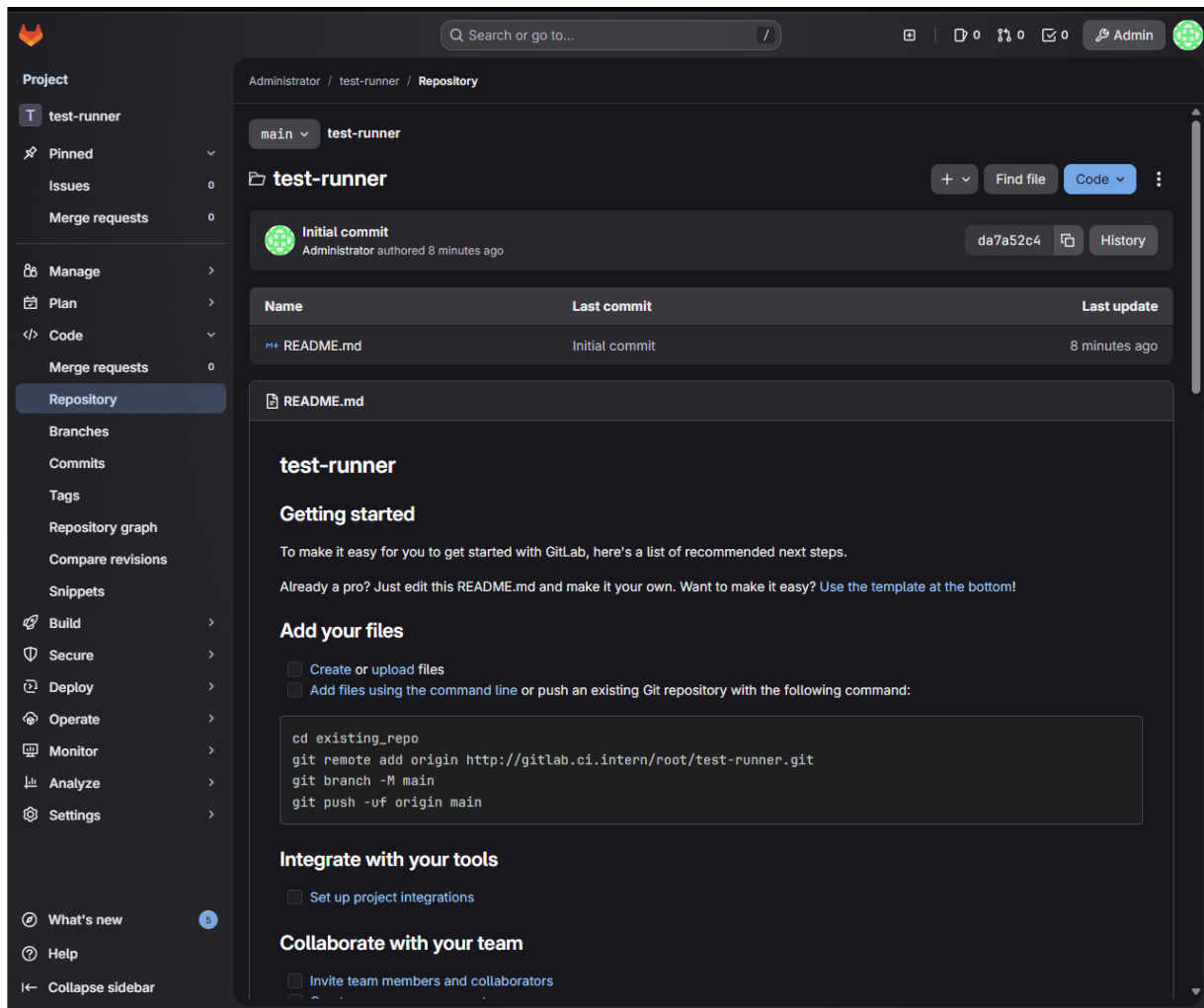


Abbildung 17: Creating a New Project

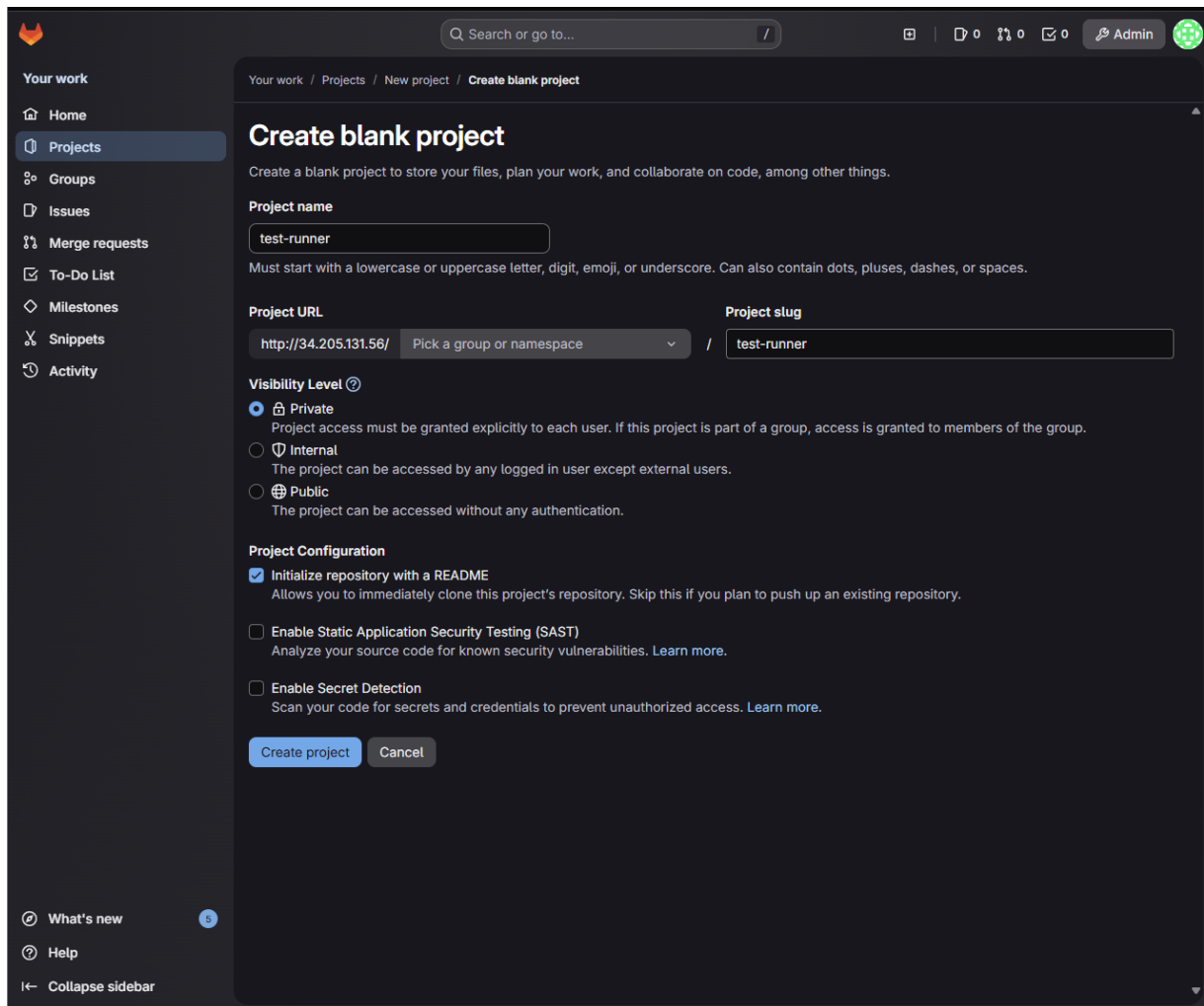


Abbildung 18: Initialized Empty Repository

1.5.2 SSH Configuration

To facilitate secure communication between the local machine and the GitLab server, we generated an SSH key pair and added the public key to the GitLab user profile.

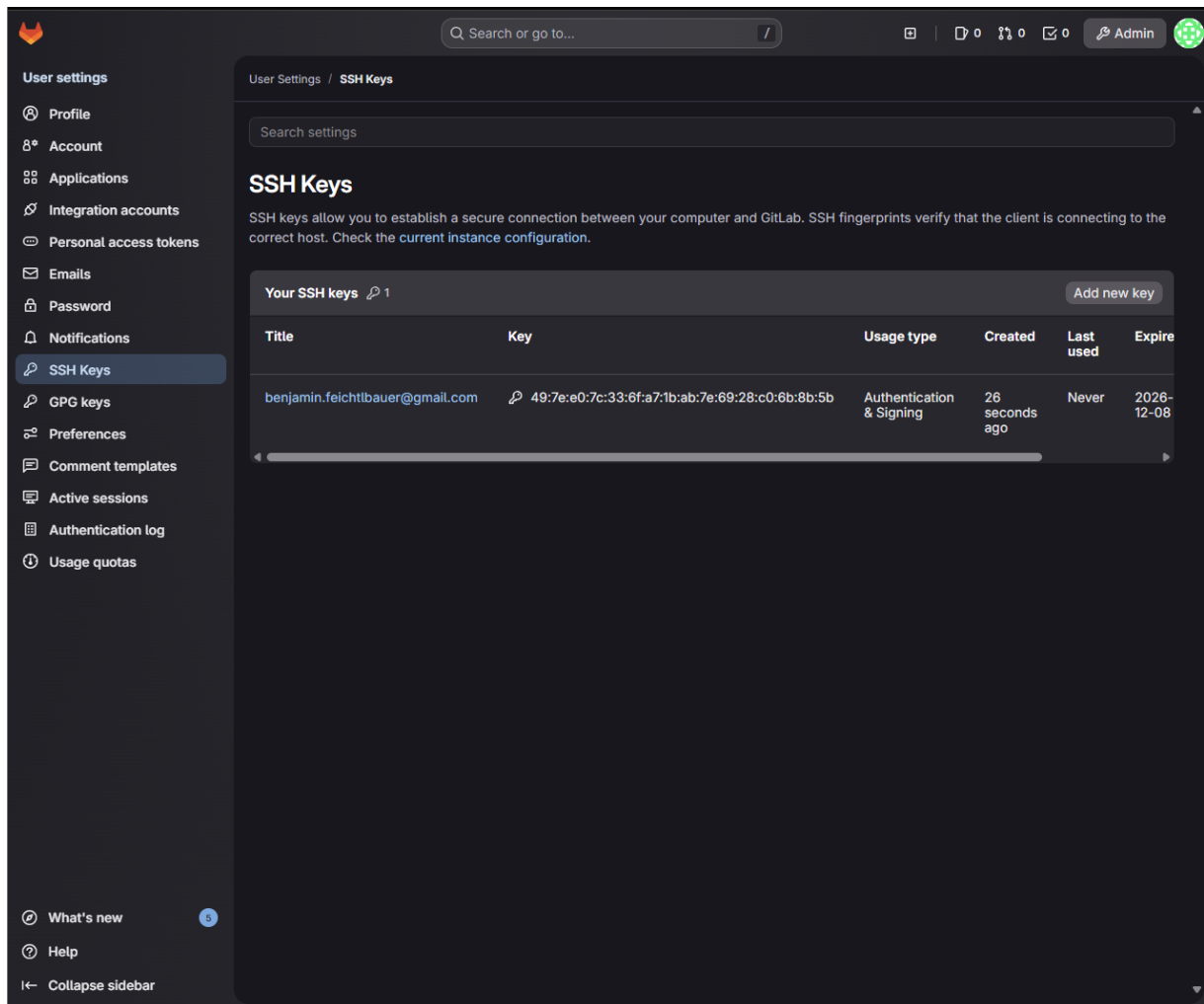


Abbildung 19: Adding SSH Key

1.5.3 Cloning and Committing

Since the local machine is outside the AWS VPC, we cloned the repository using the GitLab server's public IP address.

```
git clone http://<PUBLIC_IP>/root/test-runner.git
cd test-runner
```

We then created a test file `demo.txt` and committed it to the repository.

```
echo "test_run_$(date)" >> demo.txt
git add .
git commit -m "demo_pipeline_test"
```

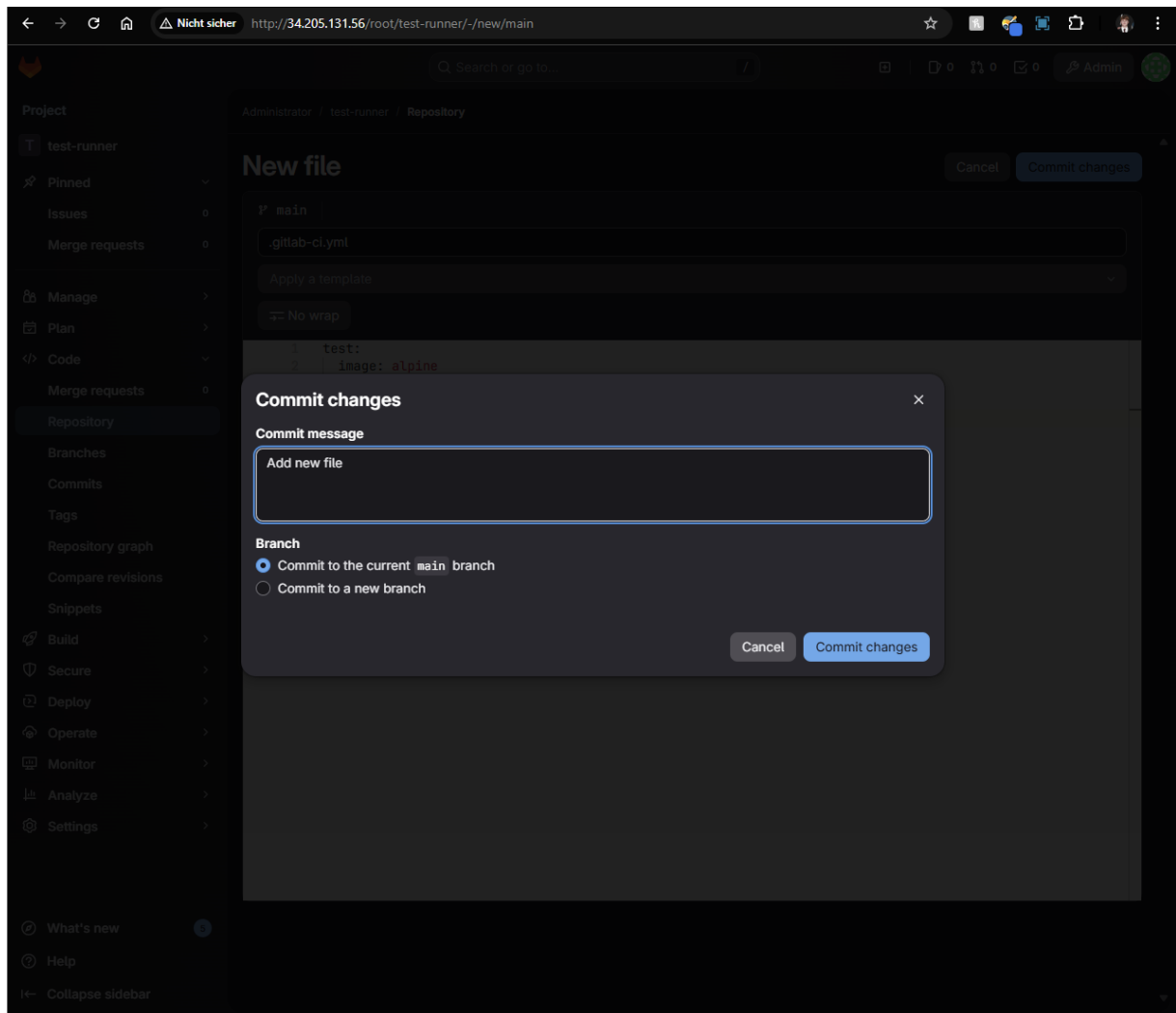


Abbildung 20: Git Commit

1.5.4 Pushing Changes

The changes were pushed to the remote repository.

```
git push
```

```
cinnamon@XPS:~/test-runner$ echo "test run $(date)" >> demo.txt
cinnamon@XPS:~/test-runner$ git add .
git commit -m "demo pipeline test"
[main 9baa5e2] demo pipeline test
1 file changed, 1 insertion(+)
create mode 100644 demo.txt
cinnamon@XPS:~/test-runner$ git push
Username for 'http://34.205.131.56': root
Password for 'http://root@34.205.131.56':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 364 bytes | 364.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To http://34.205.131.56/root/test-runner.git
   d9cb236..9baa5e2  main -> main
```

Abbildung 21: Git Push

After the push, the new file is visible in the repository, and the CI/CD pipeline is automatically triggered.

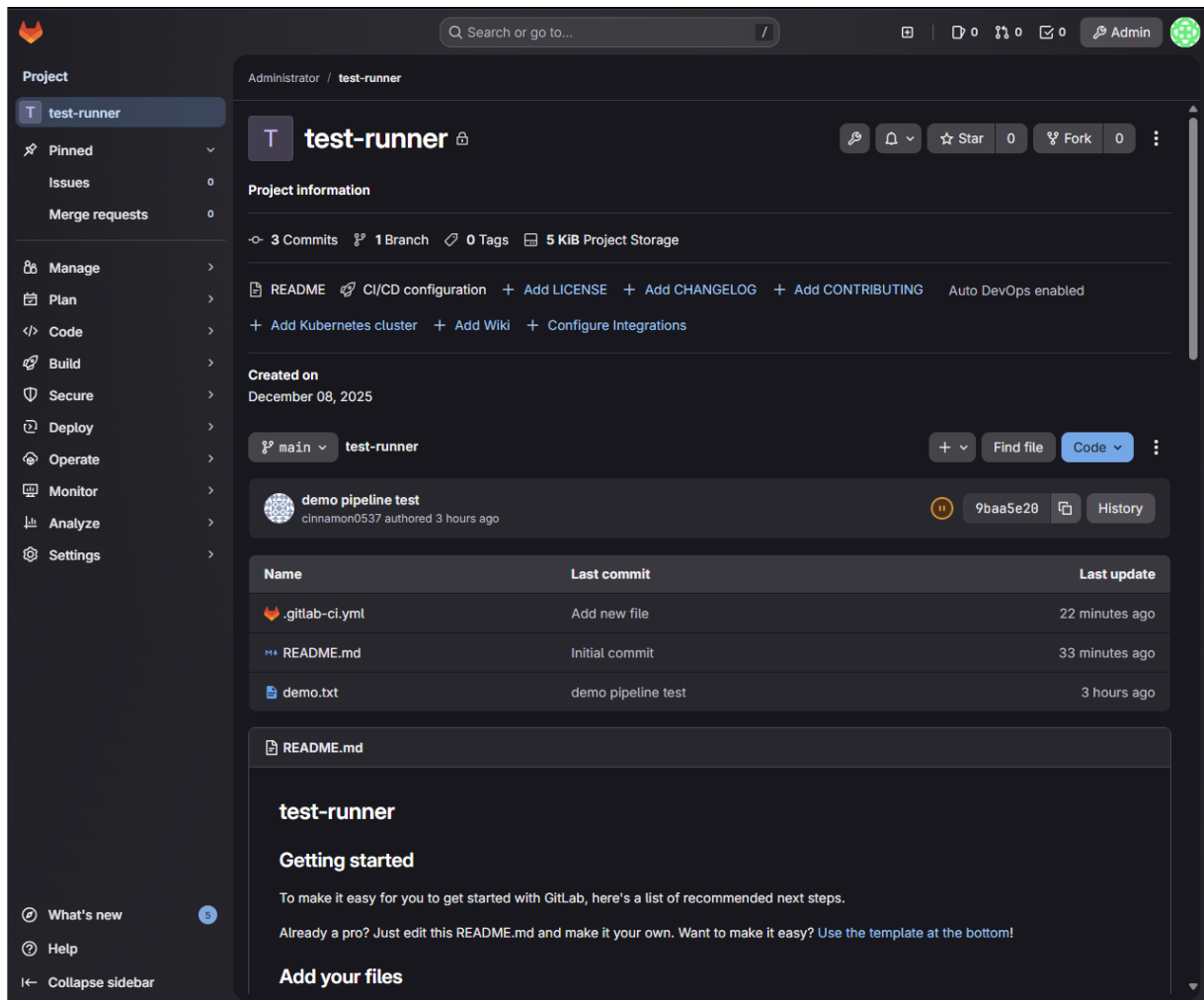


Abbildung 22: Repository after Push

2 Milestone 3

2.1 LDAP Setup – Technical Documentation

2.1.1 Goal

The goal was to implement a central user management system using LDAP for GitLab, enabling users to log in to GitLab via LDAP instead of being maintained locally. Local GitLab users should no longer be necessary, ensuring a centralized authentication source.

2.1.2 LDAP EC2 Instance Creation

Implementation

A dedicated EC2 instance was created in AWS:

- **Name:** ldap.ci.intern
- **AMI:** Ubuntu Server 22.04 LTS
- **Instance Type:** t3.micro
- **Security Group:** Separate security group, operating within the same VPC as GitLab.

Reasoning

LDAP is a core infrastructure service. It should run independently of the GitLab server and be administrable separately to reflect a realistic cloud architecture.

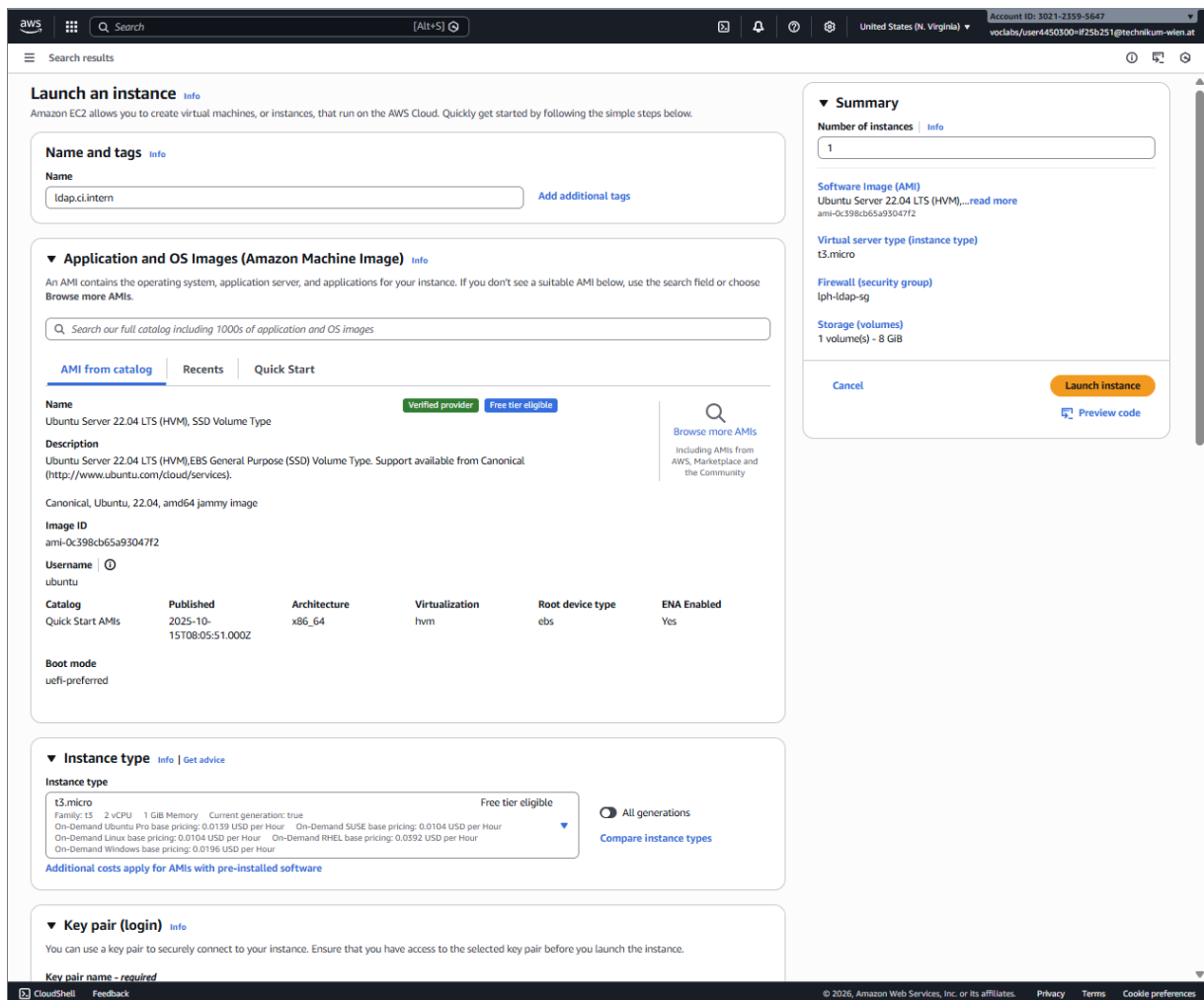


Abbildung 23: LDAP EC2 Instance in AWS

2.1.3 Security Configuration

To ensure the security of the user directory, a dedicated Security Group was configured for the LDAP instance.

- **SSH (Port 22):** Inbound traffic is restricted to the administrator's IP address for maintenance purposes.
- **LDAP (Port 389):** Inbound traffic is restricted solely to the GitLab Security Group. This configuration allows the GitLab server to authenticate users while blocking all other internal or external access attempts to the directory service.

Create security group Info

A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

Basic details Info

Security group name Info

lph-ldap-sg

Name cannot be edited after creation.

Description Info

Allows SSH access to developers

VPC Info

vpc-0456ae6dc8078ccce (lph-vpc)

Inbound rules Info

Type	Protocol	Port range	Source	Description - optional
SSH	TCP	22	Custom 84.115.226.143/32	
LDAP	TCP	389	Custom sg-0c282fb2d40e296ae	
LDAP	TCP	389	Custom sg-061d1870ca2d1f20e	
LDAP	TCP	389	Custom sg-01e7e340b978df65c	

Outbound rules Info

Type	Protocol	Port range	Destination	Description - optional
All traffic	All	All	Custom 0.0.0.0/0	

Tags - optional Info

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

No tags associated with the resource.

Create security group

Abbildung 24: LDAP Security Group Rules

2.1.4 Connection and Installation

After starting the instance, an SSH connection was established (`ssh ubuntu@<ldap-private-ip>`), follo-

wed by switching to the root user.

Installation of OpenLDAP

OpenLDAP was installed using the following commands:

```
apt update
apt install slapd ldap-utils -y
```

During installation, the LDAP admin password was set, and the base domain was configured as `ci.intern`, resulting in the Base DN `dc=ci,dc=intern`.

Reasoning

`slapd` is the LDAP server daemon, and `ldap-utils` provides necessary management tools like `ldapadd` and `ldapsearch`.

2.1.5 Base Structure Configuration

An initial structure was defined in a `base.ldif` file to establish the root domain and separate Organizational Units (OUs) for Users and Groups. This structure is essential for GitLab's integration.

```
dn: dc=ci,dc=intern
objectClass: top
objectClass: dcObject
objectClass: organization
o: CI Intern
dc: ci

dn: ou=users,dc=ci,dc=intern
objectClass: organizationalUnit
ou: users

dn: ou=groups,dc=ci,dc=intern
objectClass: organizationalUnit
ou: groups
```

This structure was applied using `ldapadd`.

2.1.6 User Creation

Users are managed centrally in LDAP. Passwords were generated using `slappasswd` (producing an SSHA hash) and users were defined in LDIF files (e.g., `benji.ldif`) with attributes like `uid`, `cn`, `sn`, and `userPassword`.

Example User:

```
dn: uid=benji,ou=users,dc=ci,dc=intern
objectClass: inetOrgPerson
objectClass: posixAccount
objectClass: shadowAccount
cn: Benji Feichtlbauer
uid: benji
...
```

2.1.7 GitLab LDAP Integration

GitLab was configured to authenticate against this LDAP server. The configuration in `/etc/gitlab/gitlab.rb` enabled LDAP and defined the connection details:

```
gitlab_rails['ldap_enabled'] = true
gitlab_rails['ldap_servers'] = {
  'main' => {
    'label' => 'LDAP',
    'host' => 'ldap.ci.intern',
    'port' => 389,
    'uid' => 'uid',
    'bind_dn' => 'cn=admin,dc=ci,dc=intern',
    'password' => 'LDAP_ADMIN_PASSWORD',
    'encryption' => 'plain',
    'base' => 'dc=ci,dc=intern'
  }
}
```

Reasoning

GitLab does not authenticate users itself but checks credentials against LDAP. Upon the first successful login, GitLab automatically creates an internal account linked to the LDAP identity.

2.1.8 GitLab Configuration Adjustments

With the introduction of LDAP, several adjustments were made to the GitLab configuration to ensure a secure and consistent workflow.

Disabling Local Sign-up

To enforce the use of the central LDAP directory, the option for local sign-ups was disabled. Users cannot register themselves; they must exist in the LDAP directory to log in.

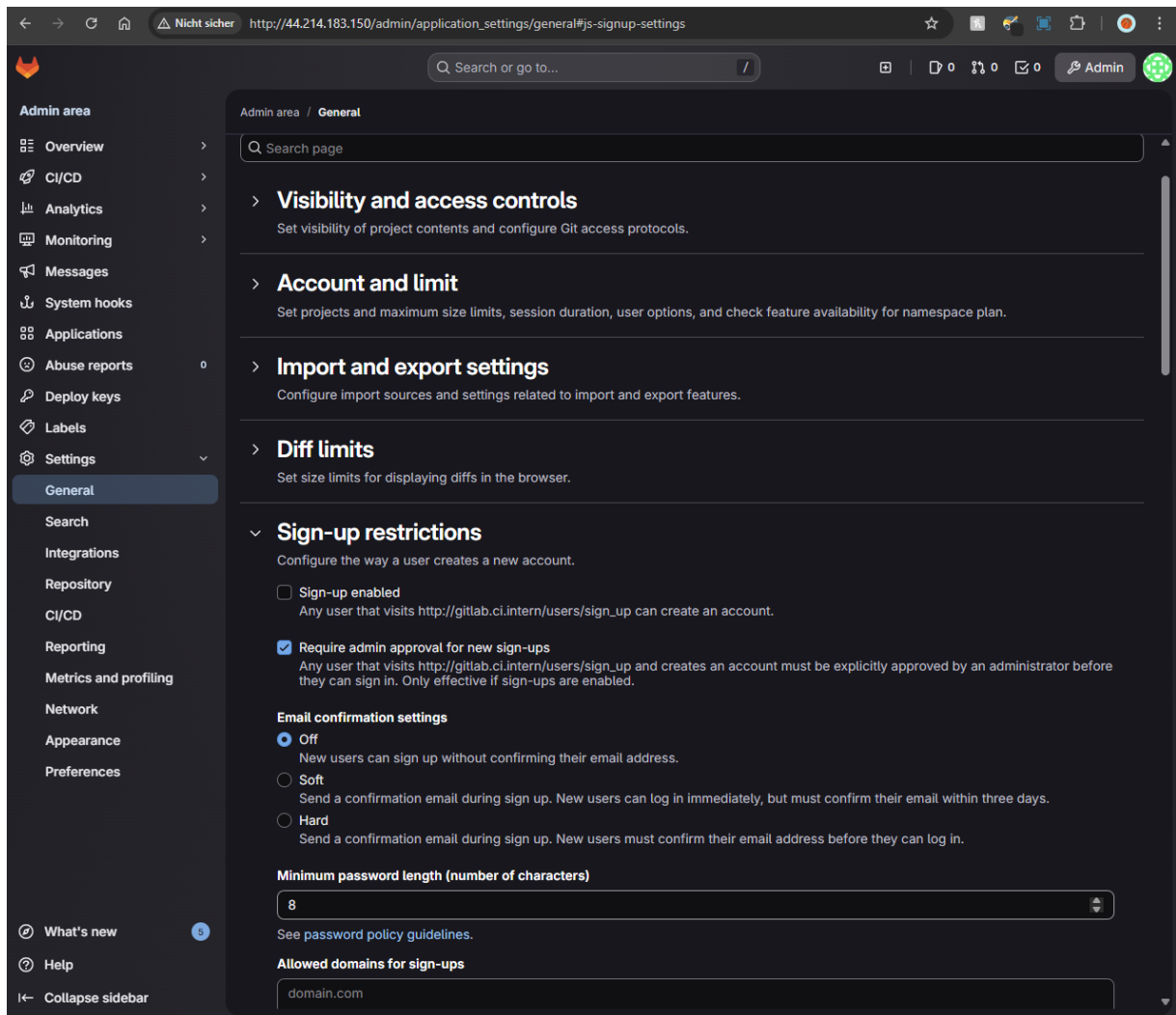


Abbildung 25: Restricted Sign-up Configuration

Project Transfer

Projects that were previously created by local users or the administrator were transferred to the new Group structure. This ensures that the newly invited LDAP users have immediate access to the codebase.

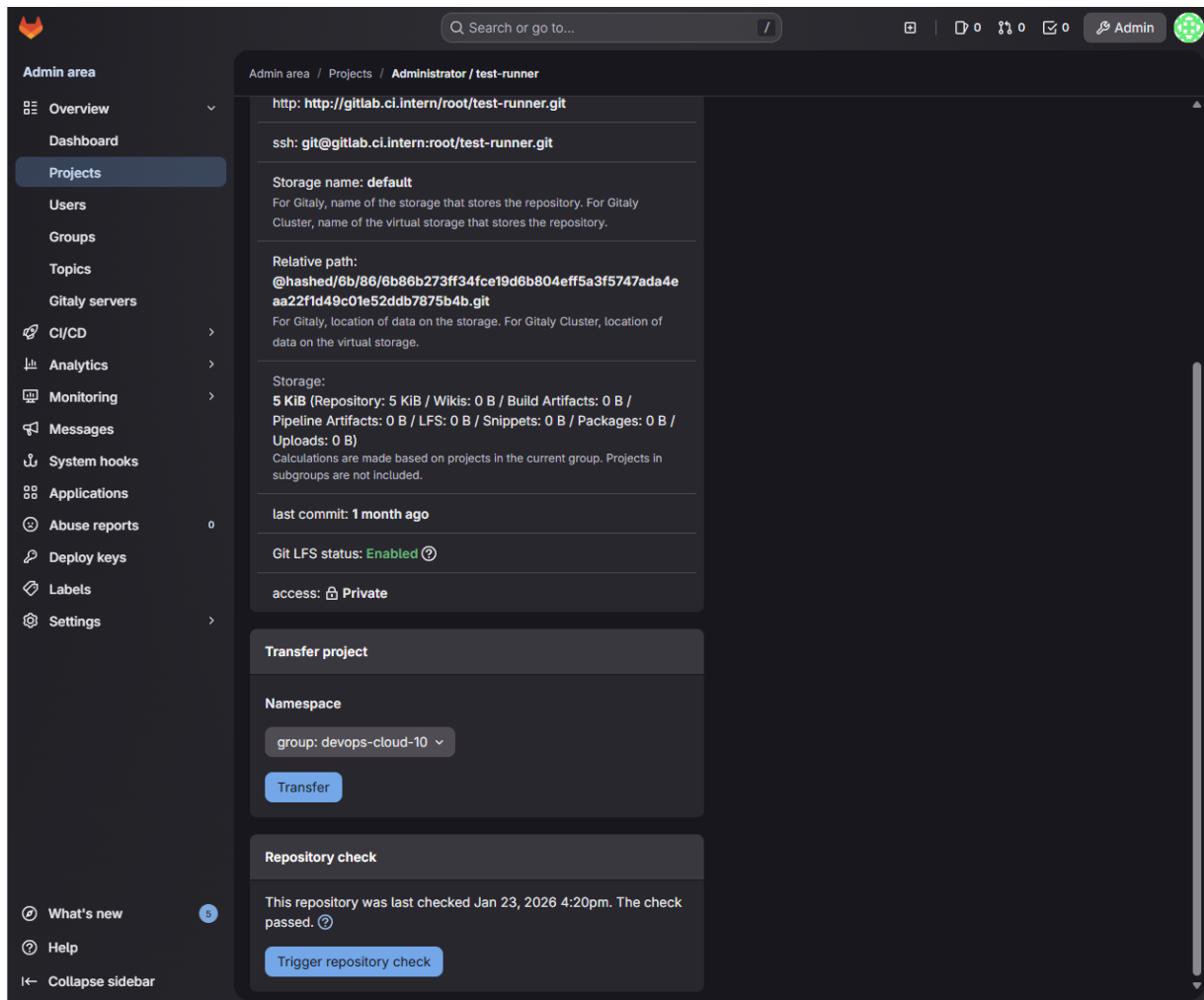


Abbildung 26: Transferring Projects to Group

2.1.9 User Onboarding and Management

Once the LDAP connection is established, users can be managed directly through GitLab's interface using their LDAP identities.

Group Configuration

A central group named `devops-cloud-10` was created to facilitate project management. All relevant projects and members are organized within this group.

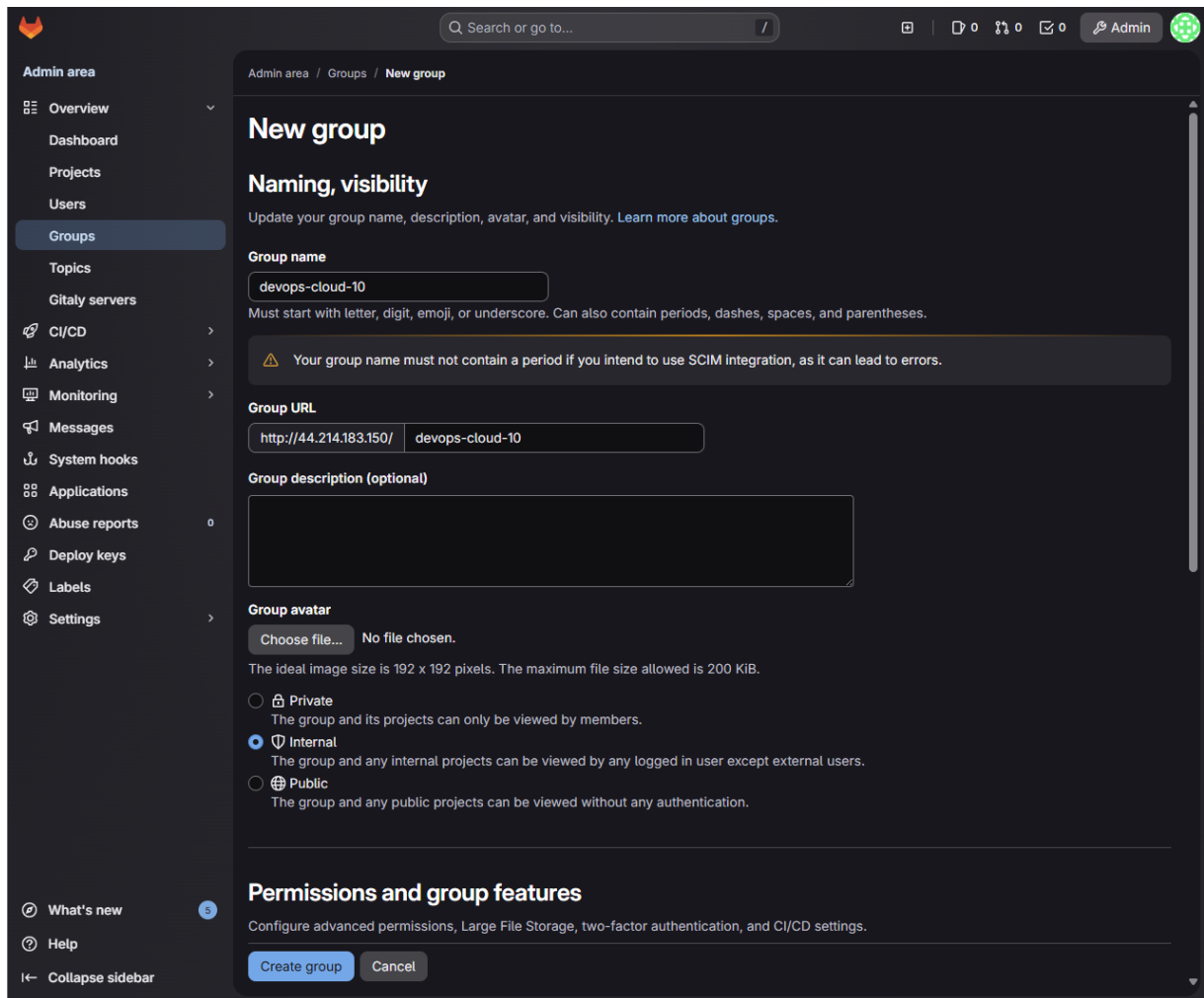


Abbildung 27: DevOps Cloud 10 Group

Inviting Users

Users do not need to be manually created in GitLab. Instead, they can be invited to Groups or Projects directly. When adding a new member, GitLab queries the connected LDAP server. This allows searching for users by their Name or UID (e.g., "benji") and assigning them permissions (e.g., Developer) before they have even logged in for the first time.

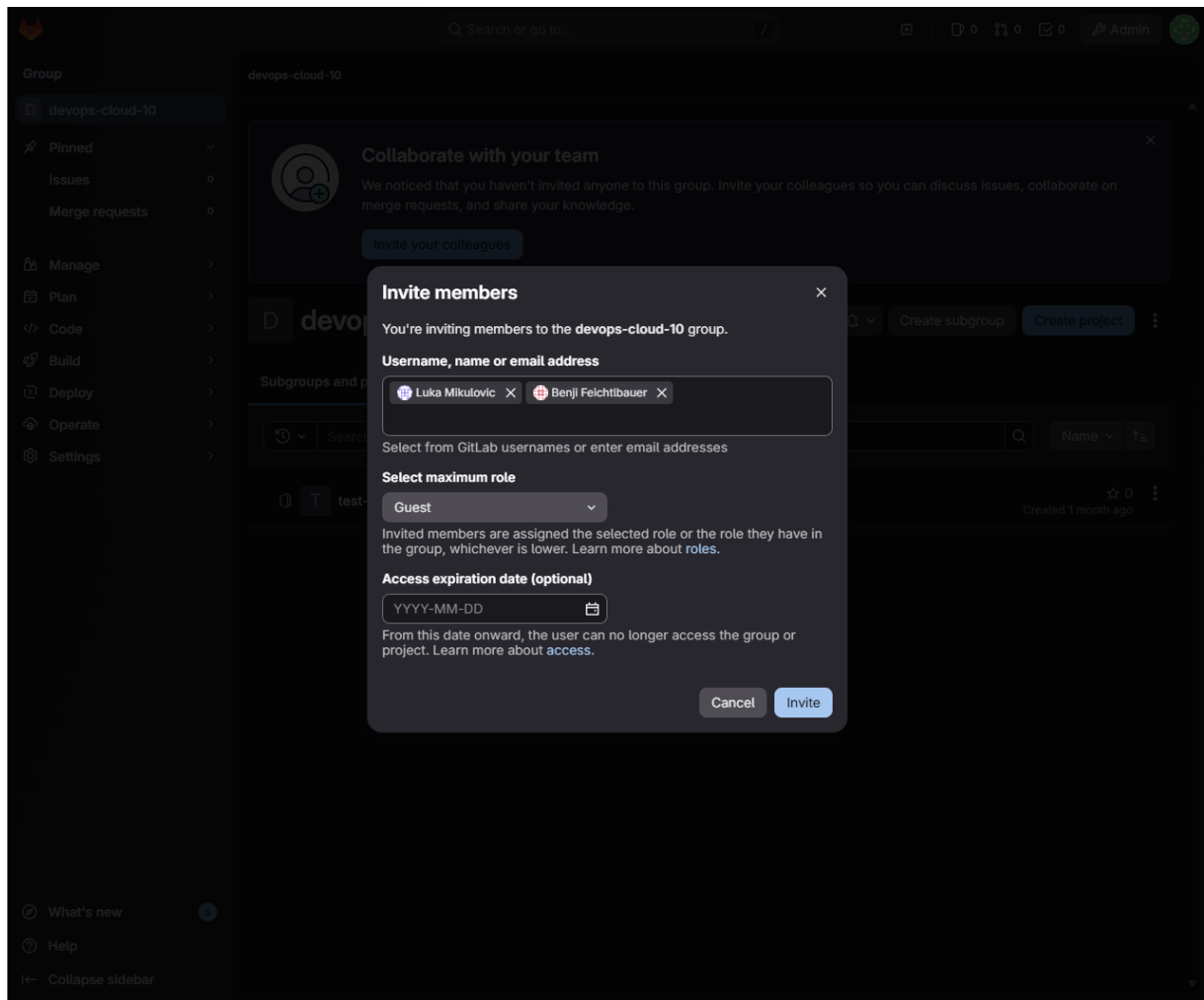


Abbildung 28: Inviting Users via LDAP Search

After the user is invited to the group, they can be managed directly through GitLab's interface using their LDAP identities.

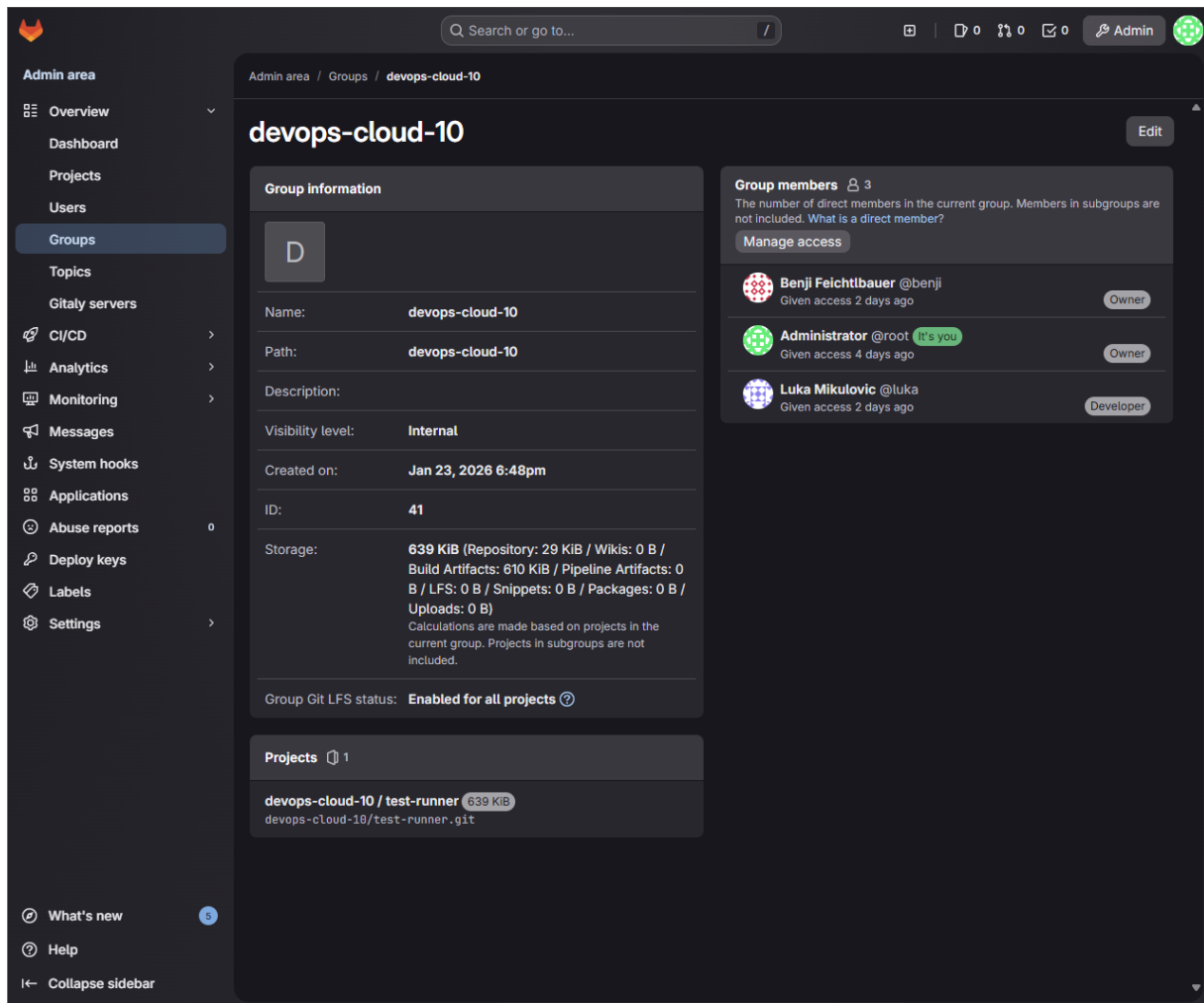


Abbildung 29: Managing Group Members

First Login

GitLab accounts for LDAP users are not created until the first successful login. Users select the "LDAP" tab on the login screen and authenticate with their directory credentials. Upon success, a GitLab user record is automatically generated and linked to the LDAP entry.

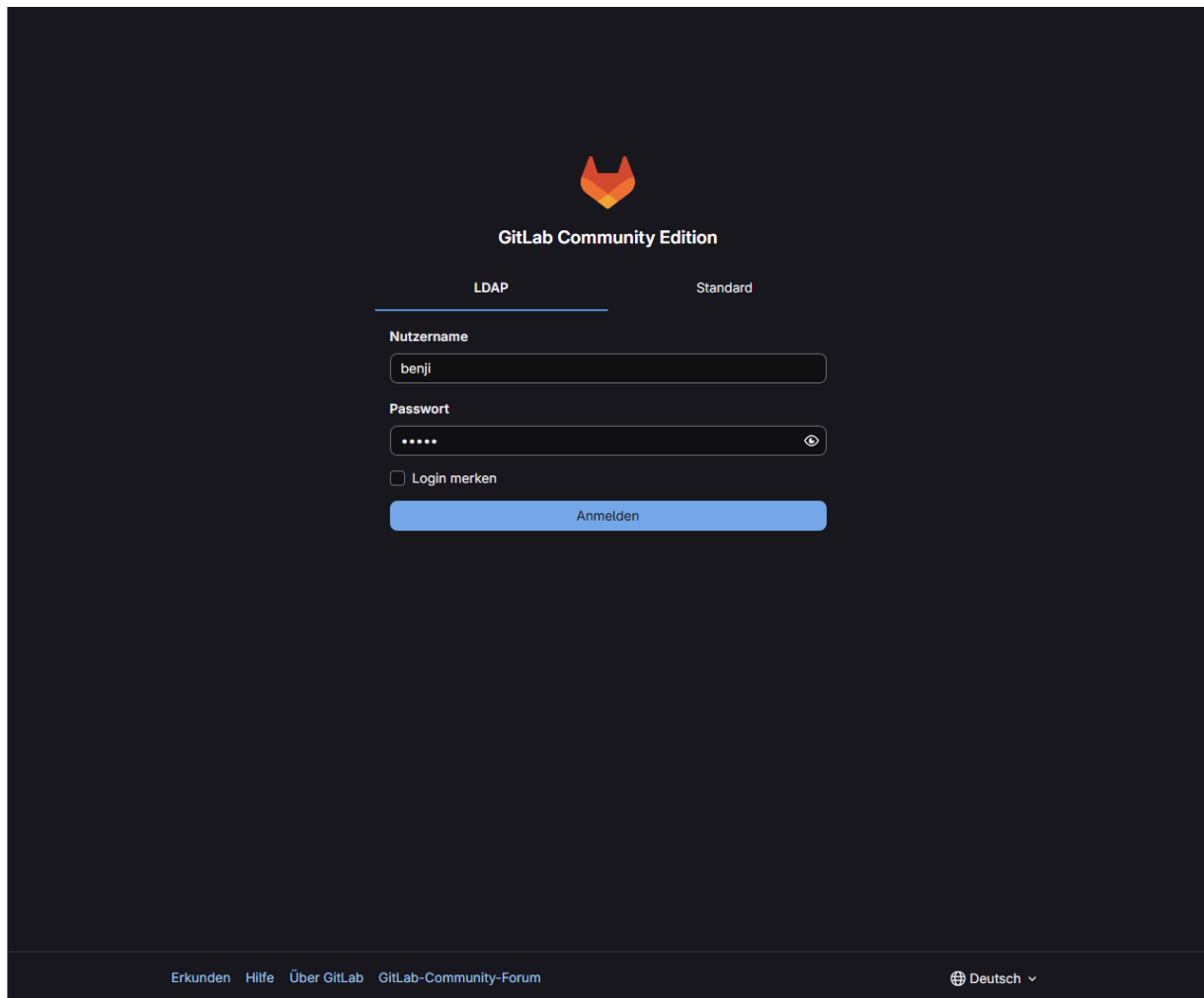


Abbildung 30: LDAP Login Screen

User Display in GitLab

After a user logs in or is added to a group, they appear in the GitLab user management area. A distinctive feature is the "LDAP" badge next to their profile, indicating that their credentials and identity are managed externally. Local password management for these users is disabled to strictly enforce the centralized authentication policy.

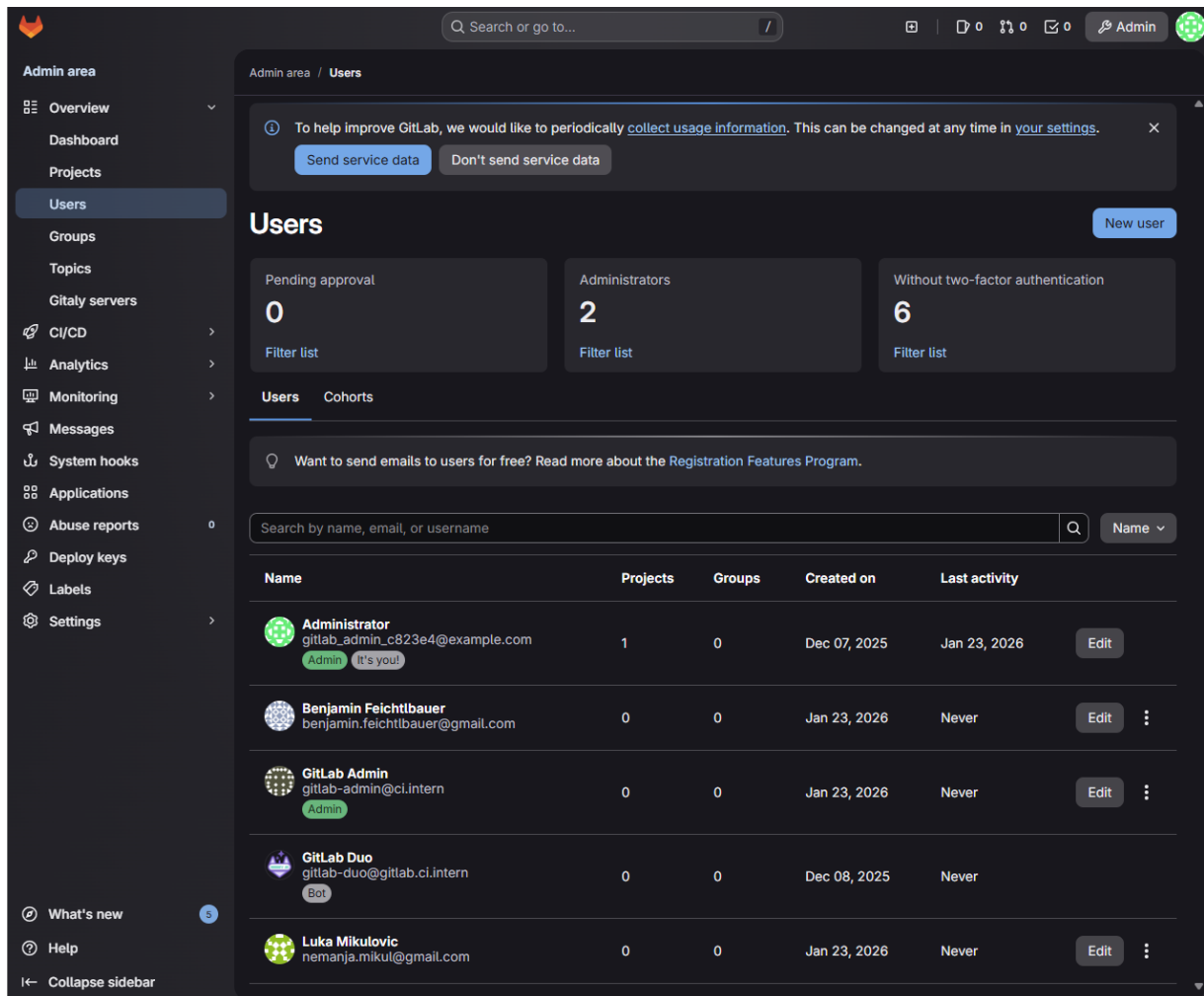


Abbildung 31: User Overview in GitLab showing LDAP users

2.1.10 Result

- Centralized LDAP server is operational.
- GitLab authentication works via LDAP.
- No local developer accounts are needed on GitLab.
- Separation of infrastructure and application is achieved.

2.2 CI/CD Pipeline Setup

2.2.1 Objective

The objective was to set up a CI/CD pipeline for a software project that runs on the deployed infrastructure, enabling automated linting, testing, building, and deployment.

2.2.2 Pipeline Decisions & Tools

- **CI/CD Platform:** GitLab CI/CD (Self-hosted).
- **Execution:** Self-hosted GitLab Runner with **Docker Executor**.
 - **Reasoning:** The Docker executor ensures that every job runs in a fresh, clean, and reproducible container, avoiding "works on my machine" issues.
- **Linting Tool: Ruff.**
 - **Reasoning:** Ruff is extremely fast and covers both linting and formatting checks. Added to the CI to "fail fast" if code quality or style standards are not met.
- **Unit Tests: Pytest.**
 - **Reasoning:** Standard test runner for Python with clear exit codes. The pipeline fails automatically if tests fail.
- **Containerization:** Docker (built from Dockerfile). Deployment as a container image is standardized and portable.
- **Container Registry: Docker Hub.**
 - **Reasoning:** Publicly accessible, easy to integrate. Authentication is handled via GitLab CI/CD variables (`DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN`).

2.2.3 Pipeline Stages

The pipeline is divided into clear stages to ensure fast feedback.

1. **Lint:** Static code analysis using Ruff.
2. **Test:** Unit tests using Pytest.
3. **Build:** Builds the Docker image.
4. **Deploy:** Pushes the Docker image to the registry.

2.2.4 Environment & Configuration

- **Base Image:** `python:3.12-slim` is used for Lint and Test stages. It is lightweight and matches the development runtime.
- **Dependencies:** Installed via `pip install -r requirements.txt` in each job to ensure a clean environment without hidden local packages.
- **Docker-in-Docker (dind):** We use the `docker:27-dind` service and `docker:27` image for the Build and Deploy stages to allow the runner to build and push Docker images from within the CI environment.

2.2.5 Branching Strategy

- **Development:** Pushes to the `develop` branch trigger the pipeline and tag the image as `$DOCKER_IMAGE:develop`.
- **Production:** Pushes to the `main` branch trigger the pipeline and tag the image as `$DOCKER_IMAGE:latest`.

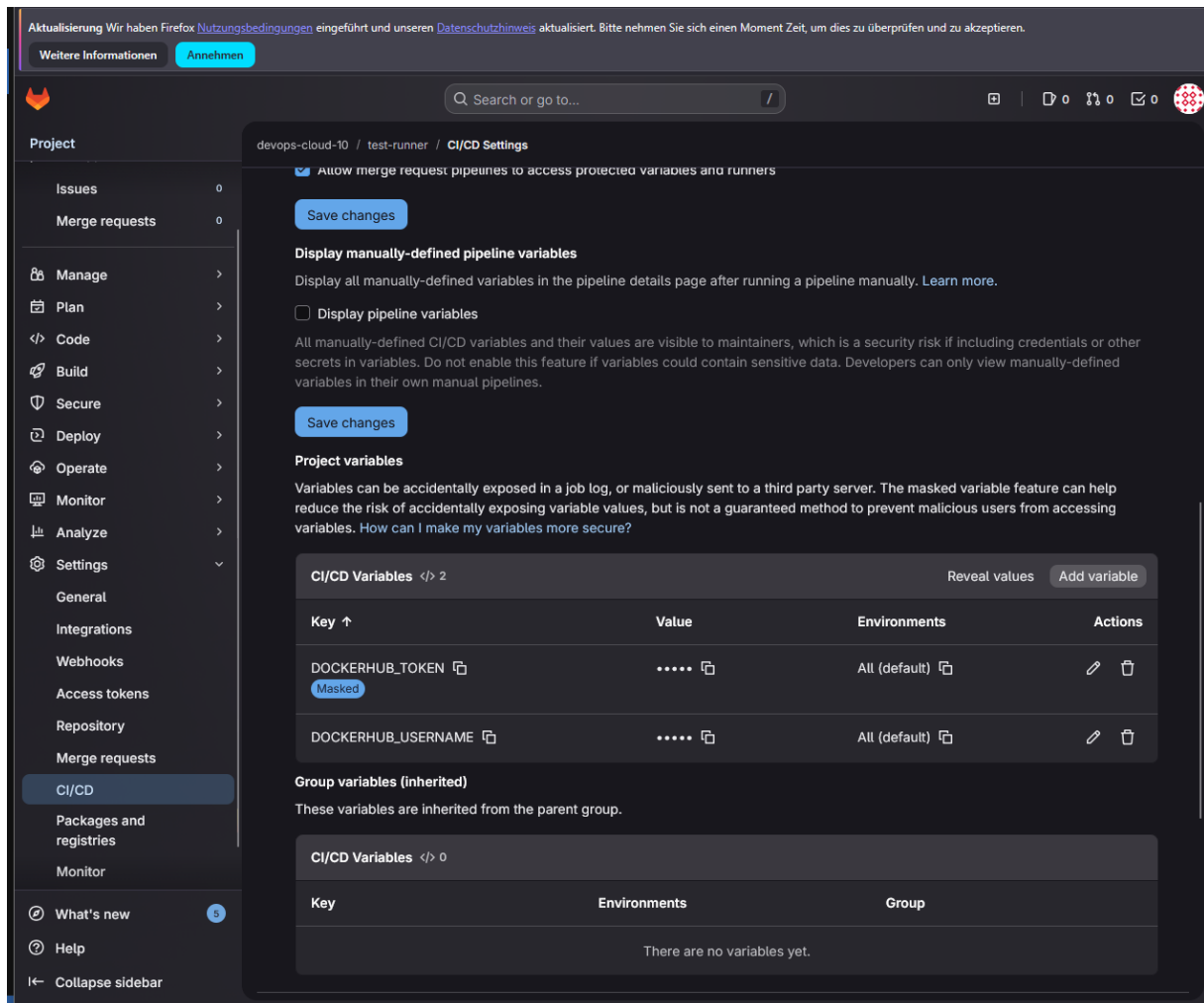


Abbildung 32: CI/CD Variables and Security Tokens

2.3 Cost Estimation

An estimation of the cloud infrastructure costs for one year was performed using the AWS Pricing Calculator.

- **EC2 Instances:** [Cost]
- **Storage (EBS):** [Cost]

- **Network Traffic:** [Cost]
- **Total Estimated Cost (1 Year):** [Total]